Secure Code Review Report

of

LMA2 Mobile Application

Date of Report Submission: 19.01.2024

Report Submitted by

Centre for Development of Advanced Computing

Legal Warning: This document contains confidential information about "**BPCL**" and can be viewed by only authorized personnel.

If you have opened this document by accident, please report to cswan[at]cdac[dot]in

REPORT DETAILS

Report Header	Secure Code Review of BPCL LMA2 Mobile Application
Version	1.0
Testing Team	C-DAC Hyderabad
Report Class	Confidential
Test Duration	17.01.2024 to 19.01.2024

LEGAL RESPONSIBILITIES

Report contents are hidden and cannot be transferred to or shared with third parties without written consent of both BPCL and CDAC.

Report contains the security vulnerabilities discovered by our team, present throughout the scanning period and known to that date. Between scanning has finished and report was delivered, new security vulnerabilities might have been reported in the world, but both parties should understand that they will not be included in the test and the testing company should not be held liable for this.

Solutions/recommendations (fixes) given in the report are for advisory purposes only. Any problems/issues resulting from the application of this solution will not be supported by CDAC. Reasonable level of support must be requested from professionals before applying the solution fixes.

Table of Contents

1. Background 4
2. Introduction 4
3. Methodology 4
4. Scope 5
5. Executive Summary of Vulnerabilities 5
6. Threats Summary 6
7. Vulnerability in Detail 7
   7.1 Client HTML5 Store Sensitive data In Web Storage 7
   7.2 Hard Coded API Keys 8
   7.3 Information Exposure Through an Error Message 9
   7.4 Insufficient Logging of Exceptions 10
   7.5 ESAPI Banned API 11
   7.6 Use of System exit 12

1. Background

CDAC was contacted by BPCL to conduct a Secure Code Review of BPCL LMA2 Mobile Application to understand the inner workings of applications, to correlate dynamic testing findings with static testing findings as well as increasing the automated test coverage that can be applied.

Efforts were placed on the identification of security weaknesses and vulnerabilities, allowing development teams to correct application defects that otherwise weaken the robustness of the application from a security perspective.

2. Introduction

This report documents the security weaknesses and vulnerabilities identified by the conducting a detailed secure source code review involving manual and automated review of the Secure Code Review of LMA2 Mobile Application

During the analysis, we have excluded the coding standards related issues and considered only the issues which impact the security posture of the application. Efforts were also put towards assessing the false positives and true positives.

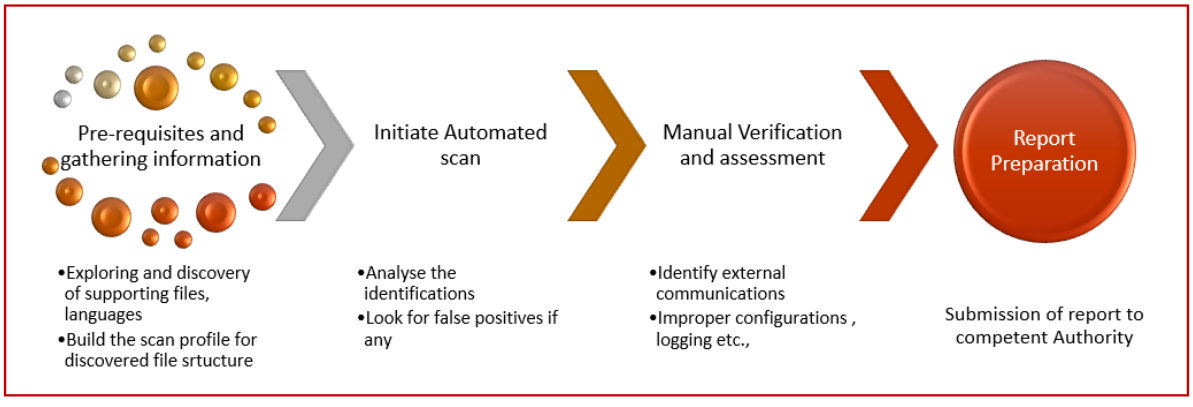
Report contains information about the identified issues and auditors' consideration of the issues as vulnerabilities. However, a detailed discussion with concerned development team may help to improve the certainty of the reported issues.

3. Methodology

As per OWASP, Secure code review is probably the single-most effective technique for identifying security bugs early in the system development life cycle. When used together with automated and manual penetration testing, code review can significantly increase the cost effectiveness of an application security verification effort.

C-DAC followed the OWASP defined source code security review to identify the security bugs and issues by performing a white box testing on the source code both manual and using automated tools.

Thus, the following is the secure code review methodology followed by us which is in-line with OWASP Standards practices.



As part of Automated Secure code review, we utilized the licensed source code review tool Checkmarx to crawl through the entire code and scan for vulnerabilities and insecure coding practices implemented. However, the automated tools may result in false positives and there are few businesses logic issues which cannot be identified by automated tools, thus, we perform a manual review on the

tool identified issues and execute in-house developed scripts to identify external communications, open ended entry points, improper logging, and improper database connectivity etc., kind of issues.

The secure code review checklist covers the following security controls and vulnerability areas.

- Data Validation
- Authentication
- Session Management
- Authorization
- Cryptography
- Error Handling
- Logging
- Security Configuration

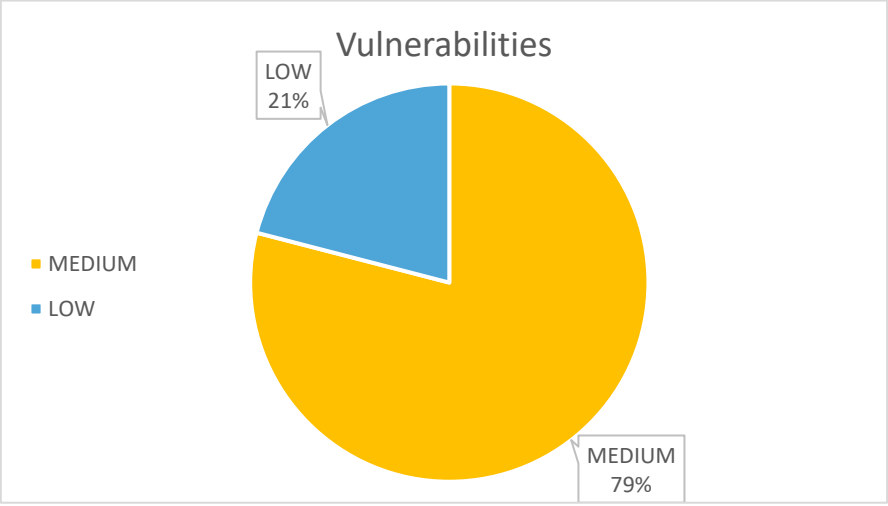
4. Scope

The scope of this activity was limited to performing the secure code review of the LMA2 Mobile Application code shared by BPCL.

Application Name	Programming Languages	Number of Lines of Code	Hash of the Code (SHA256)
LMA2 Mobile Application	React Native & Java	81305	e4eac327a725faec65d1e49e72b53e968211d8e7bb54916ad90e860ff78e6407

5. Executive Summary of Vulnerabilities

One or more Medium severity vulnerabilities are discovered in the Source Code of Application and thus the overall threat level is rated as **MEDIUM**.



6. THREATS SUMMARY

S. No.	Name Of the Vulnerability	Number of Instances	Risk Level
01	Client HTML5 Store Sensitive data In Web Storage	06	MEDIUM
02	Hard Coded API Keys	43	MEDIUM
03	Information Exposure Through an Error Message	04	LOW
04	Insufficient Logging of Exceptions	06	LOW
05	ESAPI Banned API	02	LOW
06	Use of System exit	01	LOW

7. VULNERABILITY IN DETAIL

7.1 Client HTML5 Store Sensitive data In Web Storage

Affected File	LMA/LMA2/src/api/apiClientNew.ts
Description	The application uses insecure forms of storage, which do not inherently provide any protection from unauthorized access. This data is not encrypted or sanitized before being stored, thus a malicious entity with access to the device or browser could easily retrieve the user's data.
Proof of Concept	In the below code snippet, the application stores data access_token on the client, in an insecure manner, at line 249 of LMA/LMA2/src/api/apiClientNew.ts.

```
....
248.  let access_token = await getToken();
249.  AsyncStorage.setItem('access_token', access_token);
```

Impact	A malicious entity with access to the device or browser could easily retrieve the user's data.
Recommendation	Prefer using Secure Storage modules for storing sensitive data such as auth token. Use modules such as Secure Shared Preferences or Keystore for Android and Keychain Services for iOS.
Occurrences	Similarly, 05 other occurrences of the above-mentioned vulnerability were observed at various places throughout the source code. For further details, Refer **LMA2_Detailed_Report.xlsx**.

7.2 Hard Coded API Keys

Affected File	LMA/LMA2/android/app/google-services.json
Description	The application contains API keys hard coded in the application source code. These hard coded keys are used for ensuring integrity or accessing some remote services.
Proof of Concept	In the below code snippet, the application is using a hardcoded API key.

```
....
18.  "current key": "AIzaSyClh4O5mrz-zE6VnuXq9Ndl0ReI4v23Ty8"
```

Impact	An attacker may use these keys for impersonating the application to a remote system and gain unauthorized access.
Recommendation	Prefer to not hardcode any secret data in source code, such as passwords & API keys.
Occurrences	Similarly, 42 other occurrences of the above-mentioned vulnerability were observed at various places throughout the source code. For further details, Refer **LMA2_Detailed_Report.xlsx.**

7.3 Information Exposure Through an Error Message

Affected File	LMA/LMA2/android/app/src/main/java/com/lma2/MainApplication.java
Description	The application handles exceptions in an insecure manner, including raw details directly in the error message. These exception details may include sensitive information that could leak to the users due to the occurrence of the runtime error.
Proof of Concept	In the below code snippet, the application exposes the exception details to printStackTrace

```
....
69.   } catch (ClassNotFoundException e) {
70.   e.printStackTrace();
```

Impact	Exposed details about the application's environment, users, or associated data (for example, stack trace) could enable an attacker to find another flaw and help the attacker to mount an attack. This may also leak sensitive data, e.g. passwords or database fields.
Recommendation	Do not expose exception data directly to the output or users, instead return an informative, generic error message. Log the exception details to a dedicated log mechanism.
Occurrences	Similarly, 03 other occurrences of the above-mentioned vulnerability were observed at various places throughout the source code. For further details, Refer **LMA2_Detailed_Report.xlsx**.

7.4 Insufficient Logging of Exceptions

Affected File	LMA/LMA2/android/app/src/main/java/com/lma2/FaceRdModule.java
Description	If security-critical information is not recorded, there will be no trail for forensic analysis and discovering the cause of problems or the source of attacks may become more difficult or impossible.
Proof of Concept	In the below code snippet, the security-critical event is not logged properly, therefore the error message may have some important details omitted.

```
....
63.   } catch (Exception e) {
```

Impact	In absence of details error message/ stack trace, It will be difficult to identify the root cause of the issue.
Recommendation	Use a centralized logging mechanism that supports multiple levels of detail. Be sure to set the level of logging appropriately in a production environment. Sufficient data should be logged to enable system administrators to detect attacks, diagnose errors, and recover from attacks.
Occurrences	Similarly, 05 other occurrences of the above-mentioned vulnerability were observed at various places throughout the source code. For further details, Refer **LMA2_Detailed_Report.xlsx**.

7.5 ESAPI Banned API

Affected File	LMA/LMA2/android/app/src/main/java/com/lma2/LogsUtil.java
Description	The application is using banned APIs which may lead the application vulnerable to various known exploitations.
Proof of Concept	In the below code snippet, the application usage an unsecure “exec” call.

```
....
12.  Process process = Runtime.getRuntime().exec("logcat -d");
```

Impact	Use of deprecated and unsafe libraries may leave the program vulnerable to exploitation
Recommendation	Prefer to use ESAPI libraries instead of unsafe and deprecated libraries, such as- *Executor.safeExec()* instead of *Runtime.exec(),* etc.
Occurrences	Similarly, 01 other occurrence of the above-mentioned vulnerability was observed in the source code. For further details, Refer **LMA2_Detailed_Report.xlsx**.

7.6 Use of System exit

Affected File	LMA/LMA2/android/app/src/main/java/com/lma2/MainActivity.java
Description	The application is invoking an "*exit()*" method which would immediately shut down the application.
Proof of Concept	In the below code snippet, the application invokes the exit method onClick in LMA/LMA2/android/app/src/main/java/com/lma2/MainActivity.java at line 50, which would immediately shut down the application.

```
....
50.  System.exit(1);
```

Impact	An attacker who can trigger a system exit can cause the application to terminate abruptly, resulting in a loss of availability and functionality for legitimate users.
Recommendation	Avoid using exit methods. Either handle any errors or exceptions or configure the application to terminate in a timely manner.
Occurrences	No other occurrence of the above vulnerability was observed throughout the source code.

*** End of the Report ***